

1. Definition

GPU programming is a form of heterogeneous programming that involves programming two different types of processors: the CPU and the GPU. The CPU acts as the host and controls the overall execution, while the GPU executes computationally intensive tasks in parallel.

2. Role of CPU and GPU

- CPU (Host): The CPU runs the main program, allocates and initializes memory, transfers data, starts the GPU program, and manages the overall execution.
- GPU (Device): The GPU executes the parallel portion of the program using a large number of threads and returns the results to the CPU.
- GPU memory and CPU memory are usually separate, so the host program manages data movement between them.

3. Thread organization

The GPU contains one or more processors, each capable of running hundreds or thousands of threads.

- Threads are organized into groups, often called SIMD groups.
- Threads within a group execute the same instruction on different data items.
- Threads in different groups can execute independently.
- Threads in a group may not execute in strict lockstep, but the next instruction is not executed until all threads in the group have completed the current instruction.

4. GPU memory organization

- GPU processors share a large block of memory.
- Each processor also has a small, faster memory accessible only to the threads running on that processor.
- This faster memory acts as a programmer-managed cache.
- Proper use of memory improves the performance of GPU programs.

5. Execution of GPU programs

1. The CPU allocates and initializes memory.
2. The CPU transfers the required data to GPU memory.
3. The CPU starts the GPU program.
4. The GPU executes the program using multiple threads in parallel.
5. The GPU produces the results.
6. The CPU collects and uses the results.

6. Branching and thread divergence

GPU programming faces challenges when conditional statements are used.

For example:

```
if (rank_in_gp < 16)
```

```
    my_x += 1;
```

```
else
```

```
    my_x -= 1;
```

- Threads with rank less than 16 execute the first statement.
- Threads with rank 16 or greater execute the second statement.
- Threads executing the other branch remain idle.
- After the first branch is completed, the roles are reversed.
- This is called branch divergence, and it reduces efficiency.

Therefore, GPU programmers should minimize branching whenever possible.

7. Thread scheduling

- GPUs use a hardware scheduler to schedule threads.
- The scheduler has very little overhead.
- It executes an instruction when all threads in the SIMD group are ready.

- If one group is waiting for memory or another operation, the scheduler can execute another ready group.
- Creating many SIMD groups helps keep the GPU busy.

8. Advantages of GPU programming

- Provides massive parallelism.
- Suitable for large computationally intensive tasks.
- Improves performance by executing many threads simultaneously.
- Efficient for graphics, matrix operations, and scientific computing.
- Hardware scheduling reduces the overhead of managing threads.

9. Example

Matrix multiplication is a suitable example of GPU programming. Each GPU thread can calculate one element of the result matrix, allowing many elements to be computed simultaneously.

Conclusion

Thus, GPU programming combines the control of the CPU with the massive parallel processing capability of the GPU. It is highly effective for large, uniform computational tasks, but branching and memory access must be managed carefully to achieve high performance.